

Python-Referenz

Benjamain Malicke

December 11, 2025

Contents

1	Python-Schlüsselwörter	1
2	Python String-Methoden	1
3	Python List-Methoden	2
4	Python eingebaute Funktionen	2
5	Python Funktionen	3
6	Python Klassen und Vererbung	3
7	Python Math-Funktionen	4
8	Python Dict-Methoden	5
9	Python Fehler und Ausnahmen	5
10	Python Dateioperationen	6
11	Python Kurzschreibweisen	7
12	Python Print-Formatierungen	7
13	Python Objektmethoden („toString“, „hash“, „equals“)	8
14	Python Stacks und Queues	9
15	Python Kontextmanager und with	10
16	Python wichtige Standardmodule	11
17	Python Glossar (deutsch)	11

1 Python-Schlüsselwörter

Schlüsselwort	Beschreibung
False, True, None	Boolesche Konstanten bzw. „kein Wert“. [web:12]
and, or, not	Logische Operatoren in Ausdrücken. [web:12]
if, elif, else	Bedingte Verzweigungen. [web:12]
for, while, break, continue	Schleifensteuerung. [web:12]
def, return, lambda	Funktionsdefinitionen und anonyme Funktionen. [web:12]
class, pass	Klassendefinition, Platzhalter-Anweisung. [web:12]
try, except, finally, raise	Ausnahmebehandlung. [web:12]
import, from, as	Modulimport und Aliasnamen. [web:12]
with	Kontextmanager, sorgt für sauberes Aufräumen. [web:12]
in, is	Mitgliedschafts- und Identitätsoperatoren. [web:12]
global, nonlocal	Deklaration von Variablen-Sichtbarkeiten. [web:12]
yield, async, await	Generatoren und asynchrone Programmierung. [web:12]

2 Python String-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
upper()	str	Gibt eine Version mit Großbuchstaben zurück. [web:8][web:20]	<pre>s = "abc".upper() # "ABC"</pre>
lower()	str	Gibt eine Version mit Kleinbuchstaben zurück. [web:8][web:20]	<pre>s = "AbC".lower() # "abc"</pre>
strip()	str	Entfernt Leerzeichen an beiden Enden. [web:8][web:20]	<pre>s = " hi ".strip() # "hi"</pre>
replace(old, new)	str	Ersetzt Teilstrings durch einen neuen Wert. [web:8][web:20]	<pre>s = "a-b".replace("-", "_") # "a_b"</pre>
split(sep)	list[str]	Teilt den String in eine Liste von Teilstrings. [web:8][web:20]	<pre>parts = "a,b,c".split(",") # ["a", "b", "c"]</pre>
join(iterable)	str	Verbindet Strings aus einem Iterable mit Trennzeichen. [web:8][web:20]	<pre>s = ", ".join(["a", "b"]) # "a, b"</pre>
startswith(prefix)	bool	Prüft Präfix am Stringanfang. [web:8][web:20]	<pre>ok = "hello".startswith("he") # True</pre>
endswith(suffix)	bool	Prüft Suffix am Stringende. [web:8][web:20]	<pre>ok = "test.py".endswith(".py") # True</pre>

3 Python List-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>append(x)</code>	None	Fügt ein Element am Listende an. [web:7][web:16]	<pre>1 xs = [1, 2] 2 xs.append(3) # [1, 2, 3]</pre>
<code>extend(iterable)</code>	None	Hängt mehrere Elemente an. [web:7][web:16]	<pre>1 xs.extend([4, 5]) # [1, 2 2, 3, 4, 5]</pre>
<code>insert(i, x)</code>	None	Fügt Element an Position i ein. [web:7][web:16]	<pre>1 xs.insert(1, 99) # [1, 2 99, 2, 3, ...]</pre>
<code>pop(i)</code>	obj	Entfernt und liefert Element an Index. [web:7][web:16]	<pre>1 x = xs.pop() # letztes 2 Element</pre>
<code>remove(x)</code>	None	Entfernt erstes Vorkommen von x. [web:7][web:16]	<pre>1 xs.remove(99)</pre>
<code>sort()</code>	None	Sortiert Liste in-place. [web:7][web:16]	<pre>1 xs.sort() # aufsteigend</pre>
<code>reverse()</code>	None	Kehrt die Reihenfolge um. [web:7][web:16]	<pre>1 xs.reverse()</pre>

4 Python eingebaute Funktionen

Funktion	Rückgabotyp	Beschreibung	Beispiel
<code>len(obj)</code>	int	Anzahl der Elemente einer Sequenz oder Sammlung. [web:2][web:31][web:40]	<pre>1 len([1, 2, 3]) # 3 2 len("Hallo") # 5</pre>
<code>range(stop)</code>	range	Ganzzahlfolge von 0 bis stop-1. [web:2][web:37]	<pre>1 for i in range(3): 2 print(i) # 0 1 2</pre>
<code>range(start, stop, step)</code>	range	Ganzzahlfolge mit Start, Ende (exklusiv) und Schrittweite. [web:2][web:37]	<pre>1 list(range(1, 6, 2)) # 2 [1, 3, 5]</pre>
<code>print(*args)</code>	None	Gibt Werte mit Leerzeichen getrennt aus, Zeilenumbruch am Ende. [web:2]	<pre>1 print("a", 1) # a 1 2 print("ohne", end="!")</pre>
<code>type(obj)</code>	type	Liefert den Typ eines Objekts. [web:2]	<pre>1 type(3.14) # <class ' 2 float'></pre>
<code>int(x)</code>	int	Konvertiert Wert in Ganzzahl (falls möglich). [web:2]	<pre>1 int("42") # 42</pre>
<code>float(x)</code>	float	Konvertiert Wert in Gleitkommazahl. [web:2]	<pre>1 float("3.5") # 3.5</pre>

Funktion	Rückgabotyp	Beschreibung	Beispiel
<code>str(x)</code>	<code>str</code>	String-Darstellung eines Objekts. [web:2]	<pre>1 str(10) # "10"</pre>
<code>list(iter)</code>	<code>list</code>	Erzeugt Liste aus Iterable. [web:2]	<pre>1 list("abc") # ["a", "b", "c"]</pre>
<code>dict()</code>	<code>dict</code>	Erzeugt Wörterbuch (optionale Startwerte). [web:2]	<pre>1 d = dict(a=1, b=2)</pre>

5 Python Funktionen

Element	Rückgabotyp	Beschreibung	Beispiel
<code>def name(params):</code>	beliebig	Deklariert eine benannte Funktion. [web:12]	<pre>1 def add(a, b): 2 return a + b</pre>
<code>return expr</code>	beliebig	Beendet Funktion und gibt Wert zurück. [web:12]	<pre>1 def sign(x): 2 if x > 0: 3 return 1 4 return 0</pre>
<code>lambda args: expr</code>	Funktion	Anonyme Kurzfunktion (ein Ausdruck). [web:12]	<pre>1 square = lambda x: x * x 2 square(3) # 9</pre>
<code>*args</code>	Tupel	Beliebig viele Positionsargumente. [web:12]	<pre>1 def f(*args): 2 print(args)</pre>
<code>**kwargs</code>	Dict	Beliebig viele benannte Argumente. [web:12]	<pre>1 def g(**opts): 2 print(opts["name"])</pre>

6 Python Klassen und Vererbung

Element	Rückgabotyp	Beschreibung	Beispiel
<code>class Name:</code>	Klasse	Deklariert eine neue Klasse. [web:12]	<pre>1 class Person: 2 pass</pre>
<code>def __init__(self, name):</code>	None	Konstruktor, wird beim Erzeugen aufgerufen. [web:12]	<pre>1 class Person: 2 def __init__(self, name): 3 : self.name = name</pre>

Element	Rückgabebetyp	Beschreibung	Beispiel
self	Objekt	Verweis auf die aktuelle Instanz (ähnlich this). [web:12]	<pre> 1 class Person: 2 def greet(self): 3 print(self.name) </pre>
class Sub(Base):	Klasse	Einfache Vererbung von Base. [web:26][web:32][web:38]	<pre> 1 class Student(Person): 2 def greet(self): 3 print("Hi", self.name) </pre>
super()	Objekt	Zugriff auf Methoden der Basisklasse. [web:32][web:35]	<pre> 1 class Student(Person): 2 def __init__(self, name, id): 3 super().__init__(name) 4 self.id = id </pre>
ABC, @abstractmethod	abstraktes Interface	Abstrakte Basisklassen als Interface-Ersatz. [web:12]	<pre> 1 from abc import ABC, abstractmethod 2 3 class Shape(ABC): 4 @abstractmethod 5 def area(self): 6 ... </pre>

7 Python Math-Funktionen

Funktion	Rückgabebetyp	Beschreibung	Beispiel
math.sqrt(x)	float	Quadratwurzel von x. [web:30][web:36]	<pre> 1 import math 2 math.sqrt(16) # 4.0 </pre>
math.pow(x, y)	float	x hoch y. [web:30][web:36]	<pre> 1 math.pow(2, 3) # 8.0 </pre>
math.floor(x)	int	Abrunden auf ganze Zahl nach unten. [web:27][web:30][web:36]	<pre> 1 math.floor(3.7) # 3 </pre>
math.ceil(x)	int	Aufrunden auf ganze Zahl nach oben. [web:27][web:30][web:36]	<pre> 1 math.ceil(3.1) # 4 </pre>
math.sin(x), math.cos(x), math.tan(x)	float	Trigonometrische Funktionen (Radian). [web:27][web:30][web:36]	<pre> 1 math.sin(math.pi/2) # 1.0 </pre>
math.log(x)	float	Natürlicher Logarithmus von x. [web:27][web:30][web:36]	<pre> 1 math.log(math.e) # 1.0 </pre>
math.log10(x)	float	Logarithmus zur Basis 10. [web:27][web:30][web:36]	<pre> 1 math.log10(100) # 2.0 </pre>

Funktion	Rückgabotyp	Beschreibung	Beispiel
<code>math.degrees(x)</code>	float	Wandelt Radiant in Grad um. [web:27][web:30][web:36]	<pre>1 math.degrees(math.pi) # 2 180.0</pre>
<code>math.radians(x)</code>	float	Wandelt Grad in Radiant um. [web:27][web:30][web:36]	<pre>1 math.radians(180) # 2 3.1415...</pre>
<code>math.pi</code> , <code>math.e</code>	float	Mathematische Konstanten π und e . [web:30][web:36][web:39]	<pre>1 math.pi # 3.1415... 2 math.e # 2.7182...</pre>

8 Python Dict-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>get(key, default)</code>	beliebig	Gibt Wert zu <code>key</code> oder <code>default</code> zurück. [web:45][web:50][web:55]	<pre>1 d = {"a": 1} 2 x = d.get("b", 0) # 0</pre>
<code>keys()</code>	<code>dict_keys</code>	Sicht auf alle Schlüssel. [web:43][web:45][web:50]	<pre>1 for k in d.keys(): 2 print(k)</pre>
<code>values()</code>	<code>dict_values</code>	Sicht auf alle Werte. [web:43][web:45][web:50]	<pre>1 sum(d.values())</pre>
<code>items()</code>	<code>dict_items</code>	Paare aus (Schlüssel, Wert). [web:43][web:45][web:50][web:55]	<pre>1 for k, v in d.items(): 2 print(k, v)</pre>
<code>update(other)</code>	None	Fügt Schlüssel hinzu/überschreibt vorhandene. [web:43][web:45][web:50]	<pre>1 d.update({"b": 2})</pre>
<code>pop(key)</code>	beliebig	Entfernt Schlüssel und gibt Wert zurück. [web:43][web:45][web:50]	<pre>1 v = d.pop("a")</pre>
<code>popitem()</code>	(<code>key</code> , <code>val</code>)	Entfernt letztes Paar. [web:43][web:45][web:50]	<pre>1 k, v = d.popitem()</pre>
<code>setdefault(key, default)</code>	beliebig	Gibt Wert, legt neuen an falls nicht vorhanden. [web:43][web:50]	<pre>1 d.setdefault("c", 0)</pre>

9 Python Fehler und Ausnahmen

Ausnahme	Kategorie	Typische Ursache	Beispiel
<code>SyntaxError</code>	Parserfehler	Ungültige Python-Syntax. [web:46][web:51][web:56]	<pre>1 if True print("x")</pre>
<code>NameError</code>	Laufzeitfehler	Zugriff auf nicht definierte Variable. [web:46][web:51][web:56]	<pre>1 print(x) # x nicht 2 definiert</pre>

Ausnahme	Kategorie	Typische Ursache	Beispiel
TypeError	Laufzeitfehler	Falscher Typ für Operation/Funktion. [web:46][web:51][web:56]	<pre>1 + "a"</pre>
ValueError	Laufzeitfehler	Wert hat falsches Format/Intervall. [web:46][web:51][web:56]	<pre>int("abc")</pre>
IndexError	Sequenzfehler	Index außerhalb des Bereichs. [web:46][web:51]	<pre>[1, 2][5]</pre>
KeyError	Dictfehler	Schlüssel nicht im Dict vorhanden. [web:46][web:51]	<pre>{"a": 1}["b"]</pre>
ZeroDivisionError	Mathematisch	Division durch Null. [web:46][web:51]	<pre>1 / 0</pre>

10 Python Dateioperationen

Pattern	Rückgabetyt	Beschreibung	Beispiel
with open(path, mode) as f:	Dateiobjekt	Empfohlene Art, Dateien sicher zu öffnen. [web:47][web:52][web:57]	<pre>with open("data.txt", "r") as f: text = f.read()</pre>
open(path, "r")	Dateiobjekt	Öffnet Datei nur zum Lesen. [web:47][web:52][web:57]	<pre>f = open("data.txt", "r") line = f.readline() f.close()</pre>
open(path, "w")	Dateiobjekt	Schreiben, Datei wird neu angelegt/überschrieben. [web:47][web:52][web:57]	<pre>with open("out.txt", "w") as f: f.write("Hallo\n")</pre>
open(path, "a")	Dateiobjekt	Anhängen an bestehende Datei. [web:47][web:52][web:57]	<pre>with open("log.txt", "a") as f: f.write("Eintrag\n")</pre>
f.read()	str	Liest gesamten Inhalt. [web:47][web:52][web:57]	<pre>data = f.read()</pre>
f.readline()	str	Liest eine Zeile. [web:47][web:52][web:57]	<pre>line = f.readline()</pre>
f.readlines()	list[str]	Alle Zeilen in Liste. [web:47][web:52][web:57]	<pre>lines = f.readlines()</pre>

11 Python Kurzschreibweisen

Konstruktion	Art	Beschreibung	Beispiel
a if cond else b	ternärer Ausdruck	Python-Variante des ternären Operators. [web:48][web:53][web:58]	<pre>min_value = a if a < b else b</pre>
x += 1	Inkrement	Erhöht x um 1 (statt x++). [web:49][web:54][web:59]	<pre>x = 0 x += 1 # 1</pre>
x -= 1	Dekrement	Verringert x um 1 (statt x-). [web:49][web:54][web:59]	<pre>x -= 1</pre>
[expr for x in it]	Listen-Komprehension	Kurzschreibweise für Schleife + append. [web:12]	<pre>squares = [i*i for i in range(5)]</pre>

12 Python Print-Formatierungen

Parameter	Syntax	Beschreibung	Beispiel
end=...	print(x, end="")	Kein Zeilenumbruch, benutzerdefinierter Endstring.	<pre>1 print("A", end="") 2 print("B") # AB</pre>
sep=...	print(a, b, sep="-")	Trenner zwischen Argumenten (Default: Leerzeichen).	<pre>1 print(1, 2, sep=":") # 2 1:2</pre>
file=...	print(x, file=sys.stderr)	Ausgabe in Datei/Stream statt stdout.	<pre>1 import sys 2 print("Fehler", file=sys .stderr)</pre>
f-Strings	f"var x:.2f"	Formatierte Strings mit eingebetteten Ausdrücken.	<pre>1 name = "Max" 2 print(f"Hallo {name}") # Hallo Max</pre>
:n	f"x:5"	Mindestbreite n Zeichen (rechtsbündig).	<pre>1 print(f"{42:5}") # " 42"</pre>
:<n, :>n	f"x:<5"	Links/rechtsbündig mit Breite n.	<pre>1 print(f" {123:<4} ") # 123 2 print(f" {123:>4} ") # 123 </pre>
:^n	f"x:^5"	Zentriert mit Breite n.	<pre>1 print(f" {abc:\ string^5} ") # abc </pre>
:.nf	f"pi:.2f"	n Nachkommastellen für Float.	<pre>1 pi = 3.14159 2 print(f"{pi:.2f}") # 3.14</pre>

Fortsetzung auf nächster Seite...

Parameter	Syntax	Beschreibung	Beispiel
:d, :x	f"num:d hex:08x"	Dezimal/Hexadezimal (groß/klein).	<pre> 1 n = 255 2 print(f"{n:x}") # ff 3 print(f"{n:08x}") # 000000ff </pre>
:+	f"x:+.2f"	Vorzeichen immer anzeigen (+/-).	<pre> 1 print(f"{3.14:+.2f}") # +3.14 2 print(f"{-1.5:+.2f}") # -1.50 </pre>
:	f"x: 5.2f"	Leerzeichen als Vorzeichen für positive Zahlen.	<pre> 1 print(f"{42: 6.1f}") # " 42.0" </pre>
%, b	f"0.75:% True:b"	Prozent/Boolean (1/0).	<pre> 1 print(f" {0.75:%}") # 75.00% 2 print(f"{True:b} ") # 1 </pre>

13 Python Objektmethoden („toString“, „hash“, „equals“)

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>__str__(self)</code>	str	String-Repräsentation für <code>print(obj)</code> („toString“).	<pre> 1 class Person: 2 def __str__(self): 3 return f"Person 4 ({self.name 5 })" p = Person() print(p) # Person(Max) </pre>
<code>__repr__(self)</code>	str	Offizielle Repräsentation (Debug, REPL).	<pre> 1 def __repr__(self): 2 return f"Person('{ 3 self.name}')" p # REPL: Person('Max') </pre>
<code>__hash__(self)</code>	int	Hashwert für Sets/Dicts (muss konsistent zu <code>__eq__</code> sein).	<pre> 1 def __hash__(self): 2 return hash(self. name) </pre>
<code>__eq__(self, other)</code>	bool	Gleichheitsvergleich („equals“).	<pre> 1 def __eq__(self, 2 other): 3 if not isinstance(4 other, Person): 5 return False 6 return self.name == other.name </pre>

Methode	Rückgabebetyp	Beschreibung	Beispiel
@property	Getter	Property-Decorator für read-only Attribut.	<pre> 1 class Person: 2 def __init__(self, name): 3 self._name = name 4 5 @property 6 def name(self): 7 return self._name 8 p.name # Max (kein Aufruf!)</pre>
@name.setter	Setter	Property-Setter für Attribut.	<pre> 1 @name.setter 2 def name(self, value): 3 self._name = value 4 p.name = "Anna" # Setter wird aufgerufen</pre>
__slots__ = ['name']	Optimierung	Begrenzt Attribute, spart Speicher.	<pre> 1 class Point: 2 __slots__ = ['x', 'y'] 3 def __init__(self, x, y): 4 self.x, self.y = x, y</pre>
__init__(self)	None	Konstruktor (automatisch aufgerufen).	<pre> 1 def __init__(self, name): 2 self.name = name</pre>

14 Python Stacks und Queues

Pattern	Struktur	Beschreibung	Beispiel
Stack mit Liste	list	LIFO-Stack mit append() und pop() am Ende. [web:79][web:81]	<pre> 1 stack = [] 2 stack.append(1) # push 3 stack.append(2) 4 x = stack.pop() # 2</pre>
Queue mit Liste (einfach)	list	FIFO-Queue mit append() und pop(0) (für kleine Datenmengen). [web:79]	<pre> 1 queue = [] 2 queue.append("A") 3 queue.append("B") 4 x = queue.pop(0) # "A"</pre>

Pattern	Struktur	Beschreibung	Beispiel
Stack mit deque	collections.deque	Effizienter LIFO-Stack, konstante Zeit für append() und pop(). [web:79][web:80][web:81]	<pre> 1 from collections import deque 2 stack = deque() 3 stack.append(1) 4 stack.append(2) 5 x = stack.pop()</pre>
Queue mit deque	collections.deque	Effiziente FIFO-Queue mit append() und popleft(). [web:79][web:80][web:81]	<pre> 1 from collections import deque 2 queue = deque() 3 queue.append("A") 4 queue.append("B") 5 x = queue.popleft()</pre>
Prioritäts-Queue	heapq	Kleinster Wert zuerst, für Tasks mit Priorität. [web:96]	<pre> 1 import heapq 2 pq = [] 3 heapq.heappush(pq, 5) 4 heapq.heappush(pq, 1) 5 x = heapq.heappop(pq) # 1</pre>

15 Python Kontextmanager und with

Pattern	Ressource	Beschreibung	Beispiel
Datei-Kontext	Datei	Öffnet Datei und schließt sie automatisch nach dem Block. [web:95][web:89]	<pre> 1 with open("data.txt", "r") as f: 2 data = f.read()</pre>
Eigener Kontextmanager (Klasse)	beliebig	Klasse mit __enter__ / __exit__ kapselt Setup und Aufräumen. [web:89][web:92]	<pre> 1 class Timer: 2 def __enter__(self): 3 import time 4 self.start = time.perf_counter() 5 return self 6 def __exit__(self, exc_t, exc_v, exc_tb): 7 self.end = time.perf_counter() 8 9 with Timer() as t: 10 do_work() 11 print(t.end - t.start)</pre>

Pattern	Ressource	Beschreibung	Beispiel
Kontextmanager per Dekorator	beliebig	@contextmanager aus contextlib baut Manager aus einer Funktion. [web:83][web:86]	<pre> 1 from contextlib import contextmanager 2 3 @contextmanager 4 def temp_chdir(path): 5 import os 6 old = os.getcwd() 7 os.chdir(path) 8 try: 9 yield 10 finally: 11 os.chdir(old) </pre>
with mit Lock	Thread-Lock	Z. B. threading.Lock() als Kontext, garantiert Freigabe. [web:95]	<pre> 1 from threading import Lock 2 lock = Lock() 3 4 with lock: 5 critical_section() </pre>

16 Python wichtige Standardmodule

Modul	Kategorie	Typische Verwendung
os, os.path	Betriebssystem	Pfade, Umgebungsvariablen, Dateien/Verzeichnisse verwalten. [web:84][web:96]
pathlib	Dateisystem	Objektorientierte Pfad-API, plattformunabhängig. [web:84][web:96]
sys	Interpreter	Zugriff auf Argumente, Exit-Codes, Standardströme. [web:96]
math	Mathe	Grundlegende mathematische Funktionen und Konstanten. [web:36][web:96]
statistics	Statistik	Mittelwert, Median, Varianz, einfache Statistikfunktionen. [web:84][web:90]
random	Zufall	Pseudozufallszahlen, Zufallsauswahl, Mischen von Sequenzen. [web:90][web:96]
datetime	Datum/Zeit	Datum-Uhrzeit-Objekte, Formatierung, Zeitzoneunterstützung. [web:61][web:96]
collections	Datenstrukturen	deque, Counter, defaultdict, benannte Tupel usw. [web:79][web:81][web:90]
itertools	Itertools	Generatoren für Kombinationen, Permutationen, unendliche Zähler usw. [web:87][web:90][web:93]
functools	Funktionen	Werkzeuge wie lru_cache, partial, reduce. [web:87][web:90][web:93]
json	Datenformate	JSON lesen/schreiben, z. B. für Web-APIs. [web:61][web:96]
sqlite3	Datenbank	Eingebaute SQL-Datenbank (Datei-basiert). [web:90][web:96]

17 Python Glossar (deutsch)

Begriff	Erklärung (deutsch)
Einrückung (Indentation)	Einrückungen am Zeilenanfang markieren in Python Codeblöcke (z.B. Rumpf von <code>if</code> , Schleifen, Funktionen); falsche oder fehlende Einrückung führt zu einem <code>IndentationError</code> . [web:61][web:62][web:63][web:73]
Kommentare	Kommentarzeilen beginnen mit <code>#</code> und werden vom Interpreter ignoriert; sie dienen nur der Dokumentation des Codes. [web:61][web:64]
Mehrzeilige Kommentare	Längere Erläuterungen werden meist als mehrzeilige String-Literale mit <code>"""..."""</code> geschrieben, die nicht verwendet werden und so wie Kommentare wirken. [web:64]
Variablen anlegen	Variablen entstehen beim ersten Zuweisen mit <code>name = wert</code> ; sie dienen als Behälter für Datenwerte. [web:61]
Variablennamen	Namen dürfen Buchstaben, Ziffern und Unterstrich enthalten, aber nicht mit einer Ziffer beginnen; Groß-/Kleinschreibung ist bedeutend. [web:61]
CamelCase	Benennungsschema, bei dem jedes Wort außer dem ersten mit Großbuchstaben beginnt, z.B. <code>meinWertHeute</code> . [web:61]
PascalCase	Wie CamelCase, aber auch das erste Wort beginnt mit Großbuchstaben, z.B. <code>MeinWertHeute</code> . [web:61]
snake_case	Häufige Python-Schreibweise, alle Wörter klein und mit Unterstrichen getrennt, z.B. <code>mein_wert_heute</code> . [web:61]
Mehrere Variablen zuweisen	Mehrere Namen können in einer Zeile zugewiesen werden, etwa <code>a, b = 1, 2</code> . [web:61]
Variablen ausgeben	Mit <code>print(...)</code> werden Werte auf die Konsole geschrieben, z.B. <code>print(x)</code> . [web:61]
String-Konkatenation	Zeichenketten lassen sich mit <code>+</code> oder Formatierung (z.B. f-Strings) zu längeren Strings verbinden. [web:61]
Globale Variablen	Variablen im Moduloberbereich, die in Funktionen nur mit <code>global</code> -Schlüsselwort verändert werden dürfen. [web:61]
Eingebaute Datentypen	Python stellt Basisdatentypen wie Zahlen, Strings, Listen, Tupel, Sets und Dictionaries bereit. [web:61]
Datentyp abfragen	Mit <code>type(obj)</code> lässt sich der Typ eines Wertes ermitteln. [web:61]
Datentyp setzen / Konvertierung	Funktionen wie <code>int()</code> , <code>float()</code> , <code>str()</code> oder <code>list()</code> wandeln Werte in andere Typen um. [web:61]
Zahlen: int, float, complex	Ganzzahlen (<code>int</code>), Gleitkommazahlen (<code>float</code>) und komplexe Zahlen mit Real- und Imaginärteil (<code>complex</code>). [web:61]
Zufallszahlen	Das Modul <code>random</code> erzeugt z.B. Pseudozufallszahlen mit <code>random.random()</code> oder <code>randint()</code> . [web:61]
String-Literale	Zeichenketten werden in einfachen oder doppelten Anführungszeichen notiert, z.B. <code>"Text"</code> . [web:61]
Mehrzeilige Strings	Dreifache Anführungszeichen <code>"""..."""</code> erlauben String-Literale über mehrere Zeilen. [web:61]
Strings sind Sequenzen	Strings verhalten sich wie Sequenzen von Zeichen mit Indexierung und Slicing, z.B. <code>text[0:3]</code> . [web:61]
String-Slicing / negative Indizes	Teilstrings werden mit <code>[start:stop:step]</code> erzeugt, negative Indizes zählen vom Ende. [web:61]
String-Länge	<code>len(s)</code> gibt die Anzahl der Zeichen in einem String zurück. [web:61]
String-Inhalt prüfen	Mit <code>"abc" in s</code> wird geprüft, ob eine Teilzeichenkette vorkommt. [web:61]
Formatierte Strings	Formatierung mit f-Strings, z.B. <code>f"Name: name"</code> für lesbare Ausgaben. [web:61]
Escape-Zeichen	Mit <code>{ } n</code> , <code>{ } t</code> usw. werden Sonderzeichen wie Zeilenumbruch oder Tabulator in Strings notiert. [web:61]
Boolesche Werte	<code>True</code> und <code>False</code> repräsentieren Wahrheitswerte. [web:61]
Operatoren	Rechen-, Zuweisungs-, Vergleichs-, logische, Identitäts-, Mitgliedschafts- und bitweise Operatoren steuern Berechnungen und Vergleiche. [web:61]

Begriff	Erklärung (deutsch)
Liste	Geordnete, veränderbare Sammlung von Elementen, z. B. [1, 2, 3]. [web:28][web:65]
Tupel	Geordnete, unveränderbare Sammlung von Elementen, z. B. (1, 2, 3). [web:65][web:71]
Set	Ungeordnete Sammlung eindeutiger Elemente ohne Duplikate, z. B. {1, 2, 3}. [web:65]
Dictionary (dict)	Zuordnung von Schlüsseln zu Werten, z. B. "name": "Max". [web:43][web:61]
if / elif / else	Verzweigungen, die abhängig von Bedingungen bestimmte Codeblöcke ausführen. [web:61][web:63]
Kurzformen if, if-else	Einzeilige Schreibweisen wie <code>a if cond else b</code> verkürzen einfache Bedingungen. [web:61]
while-Schleife	Wiederholt Code so lange eine Bedingung wahr ist. [web:61]
for-Schleife / range	Iteriert über Elemente eines Iterables oder Zahlenbereiche mit <code>range()</code> . [web:61]
break / continue	<code>break</code> beendet eine Schleife, <code>continue</code> überspringt die aktuelle Iteration. [web:61]
Funktionen	Mit <code>def name(...):</code> definierte wiederverwendbare Codeblöcke mit Parametern und Rückgabewert. [web:61]
*args / **kwargs	Ermöglichen eine variable Anzahl an Positions- bzw. Schlüsselwortargumenten in Funktionen. [web:61]
Lambda-Funktionen	Kurzschreibweise für anonyme Funktionen, z. B. <code>lambda x: x * 2</code> . [web:61]
Klasse / Objekt	Klassen beschreiben eigene Typen; Objekte sind Instanzen dieser Klassen mit Eigenschaften und Methoden. [web:61]
__init__	Konstruktor-Methode, die beim Erzeugen eines Objekts automatisch aufgerufen wird. [web:61]
self	Referenz auf die aktuelle Instanz innerhalb von Methoden. [web:61]
Eltern- und Kindklasse	Vererbung ermöglicht Ableitung einer Kindklasse aus einer Elternklasse, um Verhalten wiederzuverwenden und zu erweitern. [web:61]
Iterator / Iterable	Ein Iterator liefert Elemente nacheinander, ein Iterable ist jede Struktur, über die man iterieren kann (z. B. Liste). [web:61]
Module	Dateien mit Python-Code, die Funktionen, Klassen und Variablen bündeln und per <code>import</code> eingebunden werden. [web:61]
Fehlerbehandlung (try/except)	Mit <code>try</code> und <code>except</code> werden Ausnahmen abgefangen, <code>else</code> läuft ohne Fehler und <code>finally</code> immer am Ende. [web:69][web:72][web:74][web:76]
raise	Löst bewusst eine Ausnahme aus, um Fehlerzustände zu signalisieren. [web:69][web:76]